

Infix to Postfix Conversion				
Exp= K+L-M*N+(O^P)*W/U/V*T+Q				
K+L-M*N+(O^P)*W/U/V*T+Q)				
	INPUT	STACK		EXPRESSION
1	K	(		K
2	+	(+		K
3	L	(+		KL
4	-	(+ -	(-	KL+
5	M	(-		KL+M
6	*	(- *		KL+M
7	N	(- *		KL+MN
8	+	(- * +	(- +	KL+MN* -
9	(	(+(		KL+MN*-
10	O	(+(		KL+MN*-O
11	^	(+(^		KL+MN*-O
12	P	(+(^		KL+MN*-OP
13	)	(+		KL+MN*-OP^
14	*	(+*		KL+MN*-OP^
15	W	(+*		KL+MN*-OP^W
16	/	(+* /	(+ /	KL+MN*-OP^W*
17	U	(+ /		KL+MN*-OP^W*U
18	/	(+ / /	(+ /	KL+MN*-OP^W*U /
19	V	(+ /		KL+MN*-OP^W*U / V
20	*	(+ / *	(+ *	KL+MN*-OP^W*U / V /
21	T	(+ *		KL+MN*-OP^W*U / V / T
22	+	(+ * +	(+ +	KL+MN*-OP^W*U / V / T * +
23	Q	(+		KL+MN*-OP^W*U / V / T * + Q
24	)			KL+MN*-OP^W*U / V / T * + Q +
				KL-MN*OP^W/U/VT**++Q+

n=6	Array =	8	5	3	7	6	2		
		0	1	2	3	4	5		
Pass 1	gap=n/2	i=gap	j=i	A[i-gap]	temp=A[i]				
	gap=3	i=3	j=3	A[3-3]	temp=A[3]				
		i<n		A[0]	temp=7				
	while (j >= gap && A[j - gap] > temp)								
	3>=3 && A[3-3] > 7								
	3>=3 && 8>7								
	A[j] = A[j - gap]								
	A[3] = A[3 - 3]	A[3]=A[0]	A[3]=8						
		8	5	3	8	6	2		
		0	1	2	3	4	5		
	j = j - gap;	j=3-3	j=0						
	A[j] = temp;	A[0]=7							
		7	5	3	8	6	2		
		0	1	2	3	4	5		
		i++	j=i	A[i-gap]	temp=A[i]				
		i<n		A[4-3]	temp=A[4]				
	gap=3	i=4	j=4	A[1]	temp=6				
	while (j >= gap && A[j - gap] > temp)								
	4>=3 && A[4-3] > 6								
	3>=3 && 5>7	FALSE							
	A[j] = A[j - gap]	wont happen							
	A[j] = temp;	A[4]=6							
		7	5	3	8	6	2	No change	
		0	1	2	3	4	5		
		i++	j=i	A[i-gap]	temp=A[i]				
	gap=3	i=5	j=5	A[5-3]	temp=A[5]				
		i<n		A[2]	temp=2				
	while (j >= gap && A[j - gap] > temp)								
	5>=3 && A[5-3] > 2								
	3>=3 && 3>2								
	A[j] = A[j - gap]								
	A[5] = A[5 - 3]	A[5]=A[2]	A[5]=3						
		7	5	3	8	6	3		
		0	1	2	3	4	5		
	j = j - gap;	j=5-3	j=2						
	A[j] = temp;	A[2]=2							
		7	5	2	8	6	3		
		0	1	2	3	4	5		
		i++							
		i<n	FALSE						
Pass 2		7	5	2	8	6	3		
	gap=3	0	1	2	3	4	5		
	gap=gap/2	i=gap	j=i	A[i-gap]	temp=A[i]				
	gap=1	i=1	j=1	A[1-1]	temp=A[1]				
		i<n		A[0]	temp=5				
	while (j >= gap && A[j - gap] > temp)								
	1>=1 && A[1-1] > 5								
	1>=1 && 7 > 5								
	A[j] = A[j - gap]								
	A[1] = A[1 - 1]	A[1]=A[0]	A[1]=7						
		7	7	2	8	6	3		
		0	1	2	3	4	5		
	j = j - gap;	j=1-1	j=0						
	A[j] = temp;	A[0]=5							
		5	7	2	8	6	3		
		0	1	2	3	4	5		
		i++	j=i	A[i-gap]	temp=A[i]				
	gap=1	i=2	j=2	A[2-1]	temp=A[2]				
		i<n		A[1]	temp=2				
	while (j >= gap && A[j - gap] > temp)								
	2>=1 && A[2-1] > 2								
	2>=1 && 7>2								

```

void shellSort(int A[], int n) {
    int gap = n/2;
    while (gap > 0) {
        for (int i = gap; i < n; i++) {
            int temp = A[i];
            int j = i;
            while (j >= gap && A[j - gap] > temp) {
                A[j] = A[j - gap];
                j = j - gap;
            }
            A[j] = temp;
        }
        gap = gap / 2;
    }
}

```

A[j] = A[j - gap];	A[2]=A[1] A[2]=7					
A[2] = A[2- 1]	5	7	7	8	6	3
	0	1	2	3	4	5
j = j - gap;	j=2-1	j=1				
A[j] = temp;	A[1]=2					
	5	2	7	8	6	3
	0	1	2	3	4	5

here j=1 while loop will be checked again

while (j >= gap && A[j - gap] > temp)

1>=1 && A[1-1] >2
 1>=1 && 5>2

A[j] = A[j - gap];	A[1]=A[0] A[1]=5					
A[1] = A[1- 1]	5	5	7	8	6	3
	0	1	2	3	4	5
j = j - gap;	j=1-1	j=0				
A[j] = temp;	A[0]=2					
	2	5	7	8	6	3
	0	1	2	3	4	5